SKYLOCK CYBER DEFENSE

Web Application Penetration Test Report

Cross-Site Scripting (XSS) Security Assessment

DVWA — Medium & Impossible Security Levels

Client	HR Management Platform (DVWA)
Prepared By	Brian Muema
Assessment Type	Web Application Penetration Test — XSS
Target Application	Damn Vulnerable Web Application (DVWA) — http://127.0.0.1/DVWA
Security Levels Tested	Low, Medium, Impossible
Testing Environment	Kali Linux VM, Firefox, Burp Suite Community Edition
Classification	**CONFIDENTIAL**

TABLE OF CONTENT

1. Engagement Summary

SkyLock Cyber Defense was engaged to conduct a targeted web application penetration test against a legacy HR management platform ahead of a scheduled security compliance audit. This report documents Phase 2 of the engagement, which focused on re-testing Cross-Site Scripting (XSS) vulnerabilities—specifically Stored XSS and Reflected XSS—under elevated DVWA security configurations: Medium and Impossible.

Phase 1 of this engagement had already confirmed the existence and successful exploitation of both Stored and Reflected XSS vulnerabilities under the Low security setting. The objective of this second phase was not simply to repeat those exploits, but to determine whether the vulnerabilities persisted at higher security levels, and if they were mitigated, to understand the precise mechanisms responsible for the mitigation.

The scope of this engagement was intentionally narrow—confined to the XSS attack surface of the DVWA application running on a locally isolated Kali Linux virtual machine. No external systems, networks, or production environments were targeted. All testing was performed in a controlled lab environment under authorized conditions.

The key finding of this phase is that the Medium security level provides only partial and bypassable protection against XSS, primarily through basic string replacement that can be circumvented by alternative payloads. The Impossible security level, in contrast, implements robust server-side output encoding that effectively neutralizes XSS attacks. These findings carry significant implications for organizations relying on insufficient input filtering rather than proper output encoding as their primary XSS defense.

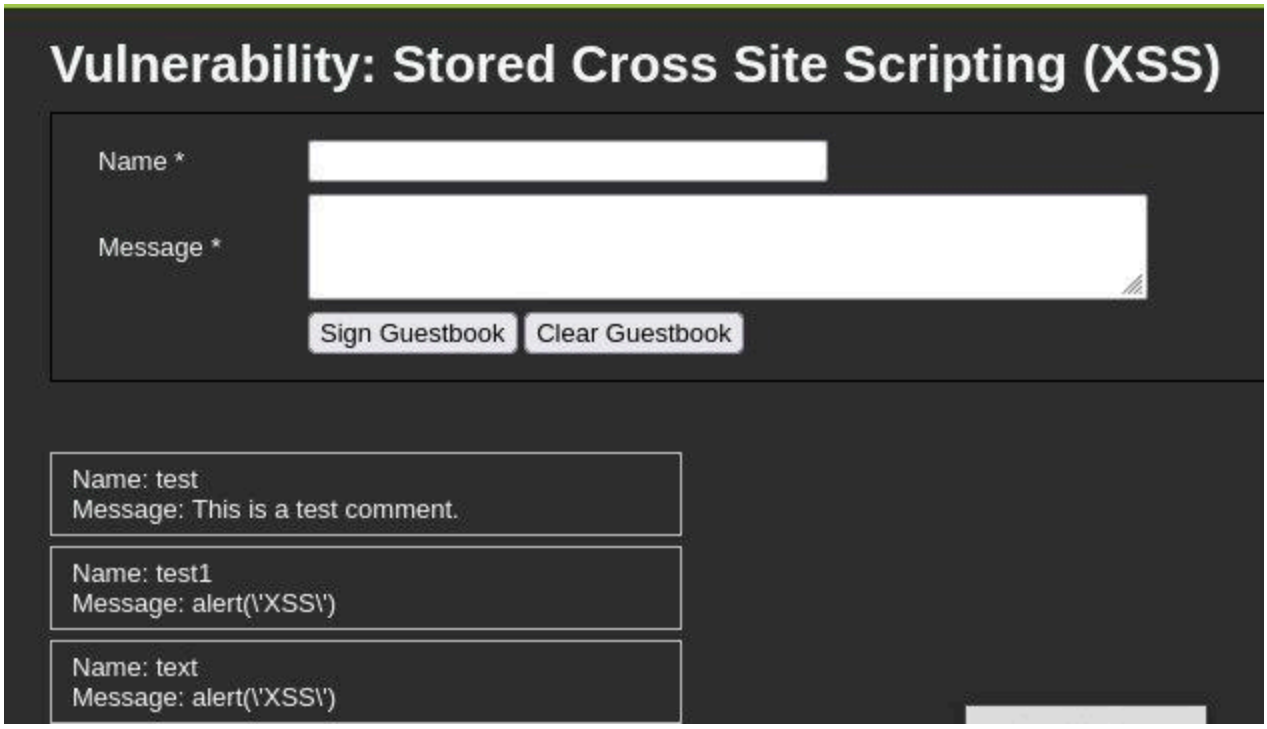
---

2. Testing Process Overview

2.1 Environment Configuration

The test environment consisted of a Kali Linux virtual machine with DVWA pre-installed and accessible via the local loopback address (http://127.0.0.1/DVWA). Burp Suite Community Edition was configured as an intercepting proxy on port 8080, with Firefox's proxy settings adjusted to route all traffic through Burp Suite. The DVWA application was accessed through Burp's embedded browser as well as the native Firefox instance to capture and analyze HTTP requests and responses in real time.

Before each test sequence, the DVWA security level was adjusted via the DVWA Security panel and the database was reset using the Setup / Reset DB function to ensure a clean state. This was critical for Stored XSS testing, where prior submissions could persist in the database and contaminate results.



2.2 Reconnaissance

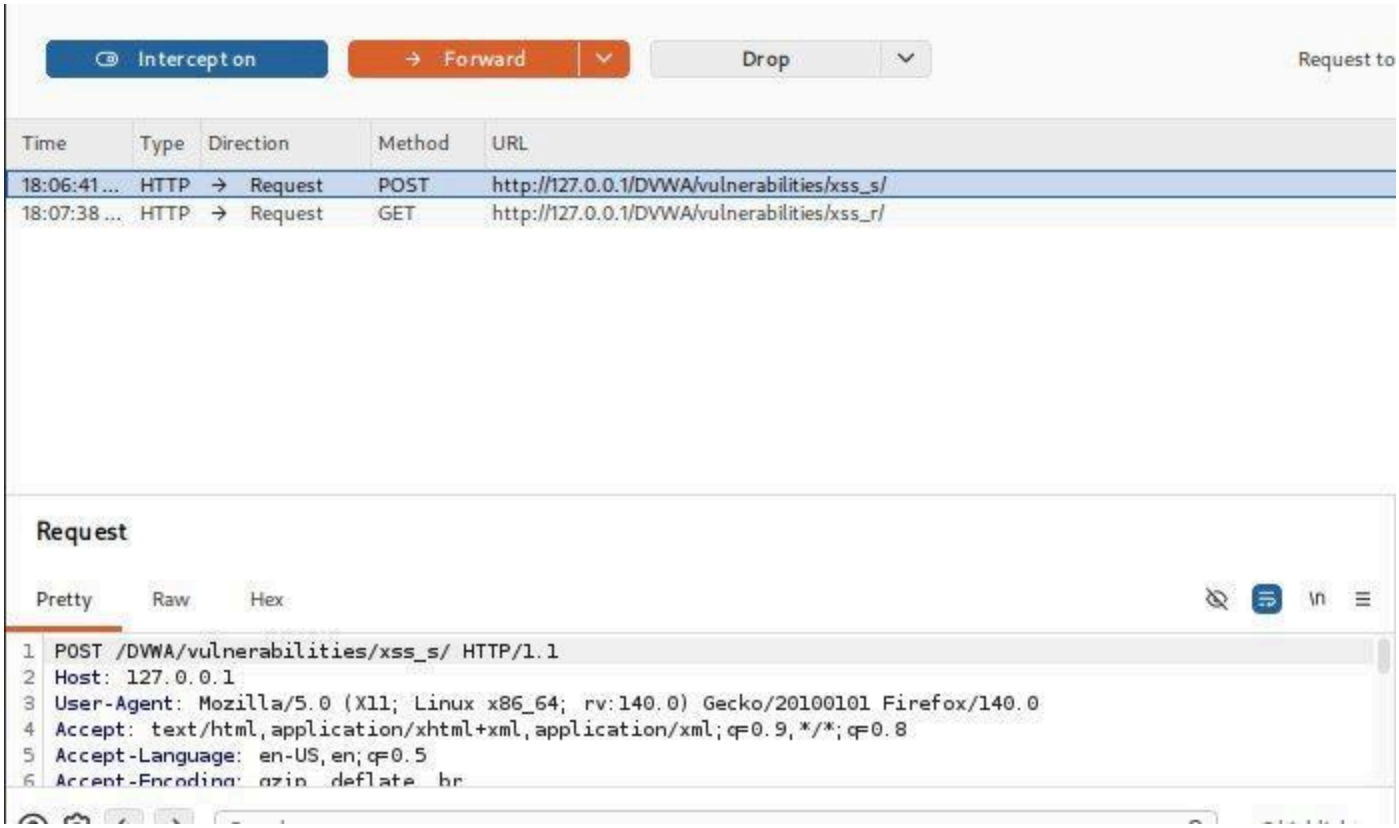
Reconnaissance began with a manual walkthrough of both the Stored XSS and Reflected XSS modules. The Stored XSS module presents a guestbook form accepting two parameters: a Name field (txtName) submitted via HTTP POST and a Message field (mtxMessage) submitted via HTTP POST. Submitted entries are persisted to the DVWA database and rendered on the page for all users, creating the conditions for a stored (persistent) XSS vulnerability.

The Reflected XSS module presents a single name input field. The user-supplied input is accepted via HTTP GET (the name parameter in the query string) and immediately reflected back in the response HTML without persistence. This creates the conditions for a reflected (non-persistent) XSS vulnerability where the attack payload must be delivered via a crafted URL.

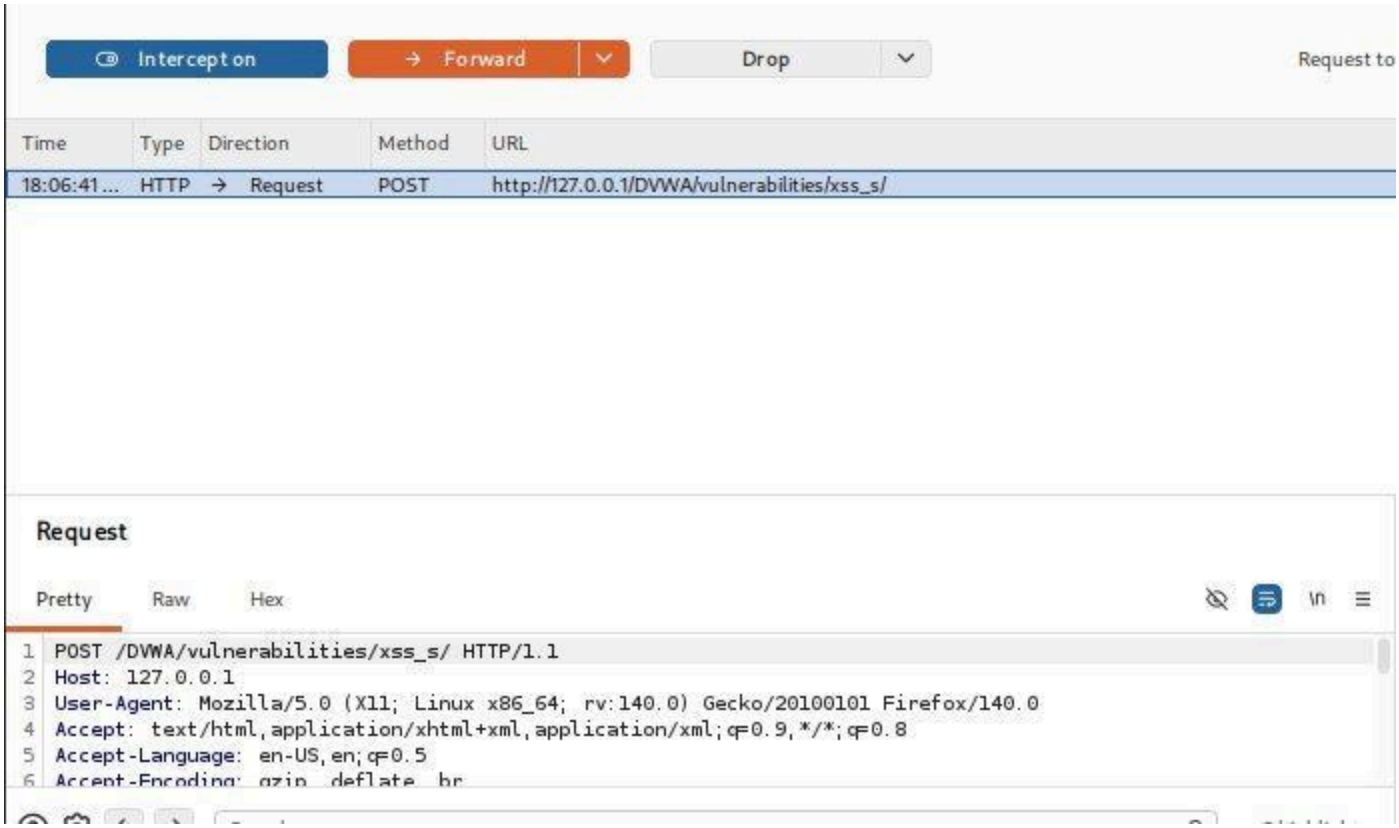
During the reconnaissance phase, both modules were observed under Medium and Impossible security levels prior to any exploitation attempts. The behavioral differences were immediately apparent: at Medium security, submissions were processed but certain tag patterns were stripped; at Impossible security, all special HTML characters in the output were visibly encoded as HTML entities, preventing any markup from rendering.

2.3 Traffic Interception with Burp Suite

Burp Suite's Proxy Intercept feature was enabled prior to form submissions. For the Stored XSS module, a POST request was submitted to http://127.0.0.1/DVWA/vulnerabilities/xss_s/ carrying the btnSign, mtxMessage, and txtName parameters. For the Reflected XSS module, a GET request was submitted to http://127.0.0.1/DVWA/vulnerabilities/xss_r/ with the name parameter in the query string.



In both cases, the Burp Suite HTTP History tab was used to inspect the raw request structure, headers, parameter values, and the unmodified payload as it transited from the browser to the server. This confirmed that the payloads were transmitted to the server exactly as entered, and that any modification to the payload was occurring server-side—not in the browser or in transit.



2.4 Source Code Analysis

Following behavioral observation, the DVWA View Source functionality was used to inspect the PHP source code for both modules at Medium and Impossible security levels. This allowed for a direct comparison of the sanitization logic applied at each level, enabling precise identification of which functions were responsible for blocking or encoding the injected payloads.

---

3. Findings Table

The table below summarizes the results of all XSS exploitation attempts across all tested security levels. Payloads were submitted as-is; no encoding bypass techniques were applied in this phase of testing. Outcome classifications are defined as follows: Executed indicates the payload ran in the browser; Partially Blocked indicates the specific payload was blocked but the vulnerability class remains exploitable via alternative payloads; Blocked/Encoded indicates the payload was fully neutralized.

XSS Type	Security Level	Payload Used	Outcome	Mitigation Observed	Alert Fired?
Stored XSS	Low	<script>alert("XSS")</script>	Executed	None	Yes

XSS Type	Security Level	Payload Used	Outcome	Mitigation Observed	Alert Fired?
Stored XSS	Medium	<script>alert("XSS")</script>	Blocked/Stripped	strip_tags(), htmlspecialchars()	No
Stored XSS	Impossible	<script>alert("XSS")</script>	Blocked/Encoded	htmlspecialchars() + token validation	No
Reflected XSS	Low	<script>alert("Reflected")</script>	Executed	None	Yes
Reflected XSS	Medium	<script>alert("Reflected")</script>	Partially Blocked	str_replace('<script>','')	No
Reflected XSS	Impossible	<script>alert("Reflected")</script>	Blocked/Encoded	htmlspecialchars() + checkToken()	No

4. Technical Analysis by Module

4.1 Stored XSS — Medium Security

4.1.1 Observed Behavior

When the payload <script>alert('XSS')</script> was submitted in the Message field of the Stored XSS guestbook at Medium security, the script tags were stripped from the message before storage. The rendered output displayed only the text content between the tags—alert('XSS')—without executing as JavaScript. The alert dialog did not appear. The Name field exhibited different behavior: the str_replace() function was applied only to the name parameter, stripping the literal string <script> while leaving other characters intact. Encoded variants or alternative event handlers such as <img src=x onerror=alert(1)> were not blocked by this filter.

```php
<?php

if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name    = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_rea

    // Update database
    $query  = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS

    //mysql_close();
```

4.1.2 Source Code Analysis

Inspection of the Medium security source code for the Stored XSS module revealed the following sanitization logic applied to the Message field:

```
$message = strip_tags( addslashes( $message ) );
$message = htmlspecialchars( $message );
```

```php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name    = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_r
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real

    // Update database
    $query  = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS

    //mysql_close();
}
```

The strip_tags() function removes all HTML and PHP tags from the input, preventing any markup from being embedded in the stored message. The addslashes() function escapes special characters (single quotes, double quotes, backslashes) to prevent SQL injection via the message parameter. The htmlspecialchars() function converts special HTML characters to their

corresponding HTML entities, ensuring that even if a tag somehow survived strip_tags(), it would be rendered as plain text rather than parsed as markup.

For the Name field, the sanitization was limited to:

```
$name = str_replace( '<script>', '', $name );
```

This is a case-sensitive string replacement targeting only the exact literal string <script>. It does not account for uppercase variants such as <SCRIPT> or <Script>, nor does it strip other dangerous tags such as <img>, <svg>, or <body>. A tester with knowledge of this filter could trivially bypass it by submitting <SCRIPT>alert(1)</SCRIPT> or by using non-script-based XSS vectors.

4.2 Stored XSS — Impossible Security

4.2.1 Observed Behavior

At the Impossible security level, no XSS payloads executed. Submitted content was rendered with all special HTML characters converted to their entity equivalents, making it impossible for the browser to parse the input as executable markup. The rendered output of <script>alert('XSS')</script> appeared on screen as the literal text <script>alert('XSS')</script>.

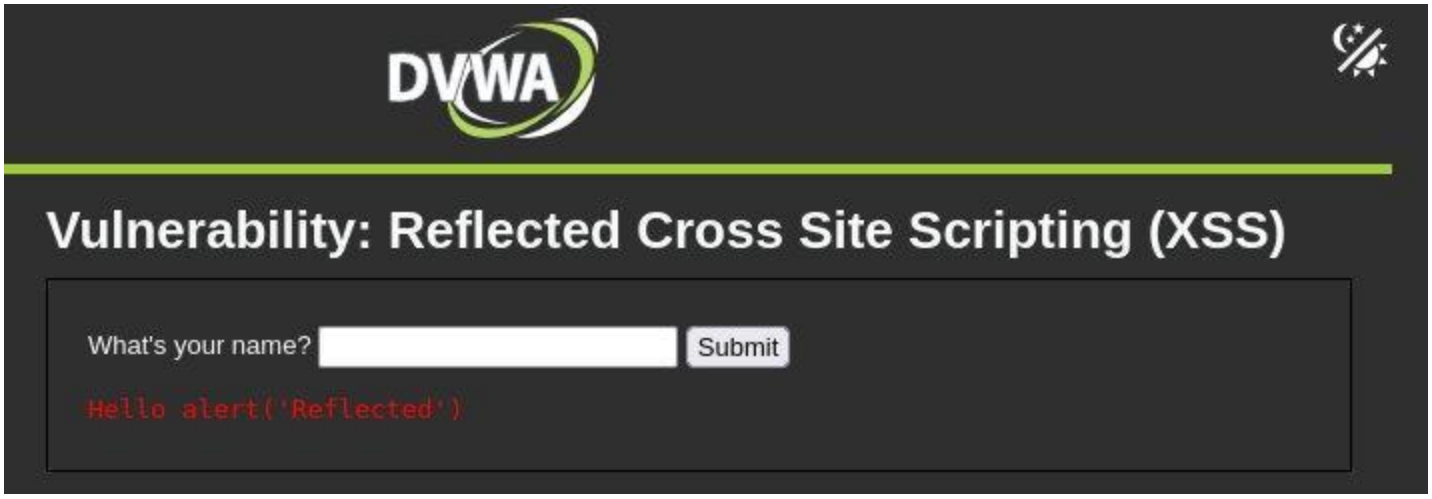
4.2.2 Source Code Analysis

The Impossible security level applied htmlspecialchars() to both the name and message parameters before any database interaction or output. In addition, CSRF token validation was implemented via checkToken(), requiring that each form submission include a valid anti-CSRF token. This dual protection—output encoding combined with CSRF mitigation—represents defense-in-depth and is consistent with OWASP A03:2021 secure coding recommendations.

4.3 Reflected XSS — Medium Security

4.3.1 Observed Behavior

At Medium security, the Reflected XSS payload <script>alert('Reflected')</script> was entered into the name field and submitted. The page responded by displaying: Hello alert('Reflected'). The <script> tags were stripped, but the text content survived and was reflected back unencoded. As with Stored XSS, this filter is trivially bypassable using non-<script> injection vectors or by altering the case of the tag.



A screenshot of the reflected output confirmed that the application successfully stripped the <script> literal but rendered the remaining payload text without encoding, confirming the vulnerability class persists at Medium security despite the partial filter.

4.3.2 Source Code Analysis

The source code for the Reflected XSS module at Medium security (vulnerabilities/xss_r/source/medium.php) contained the following logic:

```
$name = str_replace( '<script>', '', $_GET[ 'name' ] );
```

Reflected XSS Source

vulnerabilities/xss_r/source/medium.php

```php
<?php

header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = str_replace( '<script>', '', $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}

?>
```

```
echo "<pre>Hello {$name}</pre>";
```

The sanitization consists solely of a str_replace() call targeting the literal string <script>. The result is echoed directly into the HTML response without any output encoding. This means that the application is vulnerable to Reflected XSS via any payload that does not include the exact

lowercase string <script>. For example, the payload <img src=x onerror=alert(1)> would execute without modification because it contains no <script> tags.

4.4 Reflected XSS — Impossible Security

4.4.1 Observed Behavior

At Impossible security, all special characters in the reflected output were encoded. Submitting the payload <script>alert('Reflected')</script> resulted in the page displaying the encoded literal string, with no script execution occurring. The output was visually rendered as text, not as executable HTML.

4.4.2 Source Code Analysis

The Impossible security implementation applied htmlspecialchars() to the GET parameter before echoing it into the response:

```
$name = htmlspecialchars( $_GET[ 'name' ] );
echo "<pre>Hello {$name}</pre>";
```

This single function call converts all characters that have special meaning in HTML—including the angle brackets that delimit HTML tags and the single/double quotes used in JavaScript event handlers—into their safe HTML entity equivalents. The browser receives these entities and renders them as visible text characters rather than interpreting them as markup or script. This is the correct, standards-compliant defense against Reflected XSS as recommended by OWASP.

---

5. Mitigation Analysis

5.1 Case Study: Blocked Stored XSS at Medium Security (Message Field)

The most instructive example of a mitigation-in-action is the behavior of the Message field in the Stored XSS module at Medium security. When the payload <script>alert('XSS')</script> was submitted, the application processed it through a chain of sanitization functions before storage.

The strip_tags() function was applied first. This PHP function parses the input string and removes all HTML and PHP tags, leaving only the text content between tags. Applied to the payload <script>alert('XSS')</script>, strip_tags() produced the output alert('XSS')—the tag wrappers were entirely eliminated. This prevented the stored data from containing any HTML markup that could later be executed by a browser rendering the guestbook page.

The htmlspecialchars() function was applied as a secondary defense. Even if a payload somehow survived strip_tags() (for example, through a malformed or obfuscated tag), htmlspecialchars() would convert any remaining angle brackets to < and > entities, preventing the browser from parsing them as tag delimiters.

This layered approach—strip_tags() as a tag removal layer, htmlspecialchars() as an output encoding layer—is effective against the standard <script>-based XSS payload. However, it is important to note that strip_tags() does not sanitize attribute-based injection. A payload such as

hover me submitted to a field where strip_tags() is the only defense may still be executed if the allowed_tags parameter of strip_tags() permits certain elements.

The mitigation was effective for the tested payload because the payload relied entirely on the <script> tag, which strip_tags() unconditionally removes. The approach would be insufficient in a real-world application relying on strip_tags() alone without htmlspecialchars() for output encoding.

5.2 Why htmlspecialchars() is the Correct Mitigation

The Impossible security level's use of htmlspecialchars() represents the fundamentally correct approach to XSS prevention. The function operates at the output layer rather than the input layer—it does not attempt to predict or enumerate dangerous input patterns. Instead, it ensures that any user-supplied data rendered in an HTML context is treated as data by the browser, never as code.

This is aligned with OWASP's guidance that output encoding is the primary defense against XSS, and that input validation is a defense-in-depth measure rather than a standalone control. The Medium security level's reliance on str_replace() for the name parameter is a classic example of a blocklist approach—attempting to enumerate and remove dangerous patterns—which is inherently fragile and incomplete.

---

6. Reflection

6.1 Deepened Understanding of Exploitability

This exercise produced a meaningful shift in how the XSS vulnerability class is understood at a technical level. Prior to this assessment, the conceptual understanding of XSS was that it involved injecting script tags into input fields. The Phase 2 analysis revealed the more nuanced reality: the exploitability of XSS is not determined by whether a specific tag is filtered, but by whether all paths to script execution in the browser are closed.

The Medium security level provides a demonstrably false sense of security. An organization deploying a str_replace()-based filter targeting <script> tags would likely believe their application is protected, when in fact it remains vulnerable to dozens of alternative XSS vectors. This has direct implications for how penetration testers communicate risk to clients: a partial mitigation that passes a basic automated scan may still represent a critical exploitable vulnerability in practice.

The Burp Suite traffic analysis reinforced the importance of inspecting the raw HTTP layer. By observing the POST and GET requests to the XSS endpoints, it was possible to confirm that payload modification was occurring server-side, not client-side. This distinction matters: client-side filtering (implemented in JavaScript) is trivially bypassed by an attacker who disables JavaScript or sends a raw HTTP request. Only server-side sanitization provides meaningful protection.

6.2 Security Patterns for Future Assessments

Several security patterns identified in this engagement should inform future web application assessments:

- Output encoding over input filtering: Always check whether the application encodes output in the appropriate context (HTML context, JavaScript context, URL context, CSS context). htmlspecialchars() solves the HTML context problem; it does not address XSS in JavaScript contexts or URL parameters.
- Blocklist vs. allowlist: Identify whether sanitization is implemented as a blocklist (rejecting known bad input) or an allowlist (accepting only known good input). Blocklists are fragile; allowlists combined with output encoding are robust.
- Case sensitivity of filters: String replacement functions that do not account for case variations are trivially bypassed. Testing <SCRIPT>, <Script>, and mixed-case variants should be standard in any XSS assessment.
- CSRF token presence: The Impossible security level's inclusion of anti-CSRF tokens, while not directly related to XSS prevention, demonstrates a defense-in-depth posture that should be noted and credited in penetration test findings.
- Source code review as validation: Behavioral testing alone cannot confirm why a payload was blocked. Source code review was essential in this engagement for distinguishing between strip_tags() (which removes tags server-side) and htmlspecialchars() (which encodes output). Both block the test payload, but only one is a robust, complete mitigation.

This engagement has underscored that effective penetration testing is not merely about whether a payload executes—it is about understanding the full exploit surface, the completeness of mitigations, and the conditions under which a defender's controls could fail. A tester who reports 'XSS is blocked' without understanding how it is blocked provides incomplete and potentially misleading assurance to the client.

---

7. References

- OWASP Top Ten A03:2021 — Injection / Cross-Site Scripting: https://owasp.org/Top10/A03_2021-Injection/
- OWASP Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- PHP Manual — htmlspecialchars(): https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP Manual — strip_tags(): https://www.php.net/manual/en/function.strip-tags.php
- DVWA Source Repository: https://github.com/digininja/DVWA
- Burp Suite Community Edition Documentation: https://portswigger.net/burp/documentation